

CartDAOImpl.java

```
package com.harisudhan.harisudhanmart.dao;

import com.harisudhan.harisudhanmart.model.CartItem;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import javax.sql.DataSource;

public class CartDAOImpl implements CartDAO {

    private final DataSource dataSource;

    public CartDAOImpl(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public CartItem upsert(CartItem item) throws SQLException {
        Optional<CartItem> existing = findByUserAndProduct(item.getUserId(), item.getProductId());
        if (existing.isPresent()) {
            updateQuantity(existing.get().getId(), item.getUserId(),
                existing.get().getQuantity() + item.getQuantity());
            existing.get().setQuantity(existing.get().getQuantity() + item.getQuantity());
            return existing.get();
        }
        String sql = "INSERT INTO cart_items (user_id, product_id, quantity) VALUES (?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setLong(1, item.getUserId());
            ps.setLong(2, item.getProductId());
            ps.setInt(3, item.getQuantity());
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) {
                    item.setId(keys.getLong(1));
                }
            }
            return item;
        }
    }

    @Override
    public List<CartItem> findByUser(long userId) throws SQLException {
        String sql = "SELECT * FROM cart_items WHERE user_id = ?";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                List<CartItem> items = new ArrayList<>();
                while (rs.next()) {
                    items.add(map(rs));
                }
                return items;
            }
        }
    }

    @Override
```

```

public Optional<CartItem> findByUserAndProduct(long userId, long productId) throws SQLException {
    String sql = "SELECT * FROM cart_items WHERE user_id = ? AND product_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, userId);
        ps.setLong(2, productId);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? Optional.of(map(rs)) : Optional.empty();
        }
    }
}

@Override
public boolean updateQuantity(long id, long userId, int quantity) throws SQLException {
    String sql = "UPDATE cart_items SET quantity = ? WHERE id = ? AND user_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, quantity);
        ps.setLong(2, id);
        ps.setLong(3, userId);
        return ps.executeUpdate() > 0;
    }
}

@Override
public boolean remove(long id, long userId) throws SQLException {
    String sql = "DELETE FROM cart_items WHERE id = ? AND user_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.setLong(2, userId);
        return ps.executeUpdate() > 0;
    }
}

@Override
public boolean clear(long userId) throws SQLException {
    String sql = "DELETE FROM cart_items WHERE user_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, userId);
        return ps.executeUpdate() >= 0;
    }
}

private CartItem map(ResultSet rs) throws SQLException {
    CartItem item = new CartItem();
    item.setId(rs.getLong("id"));
    item.setUserId(rs.getLong("user_id"));
    item.setProductId(rs.getLong("product_id"));
    item.setQuantity(rs.getInt("quantity"));
    Timestamp ts = rs.getTimestamp("created_at");
    if (ts != null) {
        item.setCreatedAt(ts.toLocalDateTime());
    }
    return item;
}
}

```